

```
}
```

- 説明:
  - ◇ build() は MaterialApp ウィジェットを返し,アプリの UI 全体を定義する.
  - ◇ MaterialApp の中に Scaffold,Center,Text などのウィジェットが含まれる.
  - ◇ この構造がウィジェットツリーを形成し,UI のレイアウトを決める.

b. BuildContext とは？

- BuildContext は,ウィジェットがウィジェットツリーのどこにあるかを示す情報を提供する.
- 継承されたウィジェットにアクセスしたり,ナビゲーションを管理したりできる.
- build() メソッドの必須パラメータである.

c. build()が UI を更新し続ける仕組み

- Flutter は UI の変化を検出すると build() を再び呼び出す.
- UI の最新状態を確実に表示するために build() を利用する.
- Flutter は宣言的 UI モデルを採用しており,手動で UI を変更するのではなく,再構築する.

d. 比較：Java における UI の構築方法との違い

- Java (Android) では,通常,UI の更新は View オブジェクトの変更によって行われる.
- Flutter では,UI は宣言的に構築され,build() を使って再構築される.
- Flutter のアプローチにより,UI の更新が構造化され,予測しやすくなる.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- 主な違い

| 機能     | Java (Android)           | Flutter (Dart)            |
|--------|--------------------------|---------------------------|
| UI の構造 | XML や Java コードで View を作成 | build() メソッドでウィジェットツリーを返す |
| 状態管理   | View を手動で更新              | 状態が変わると UI が自動更新          |
| 柔軟性    | View 階層を変更する必要がある        | build() によりシンプルに管理        |

ステップ 7：Flutter における return の役割

a. build()における return の役割とは？

- build() メソッド内で return を使って,ウィジェットが表示する内容を指定する.

- レンダリングのためにウィジェットを Flutter に送り返す.
- Flutter は return を使って UI 要素を構築し,動的に表示する.

b. return で返されるウィジェットの理解

| ウィジェット             | 使用目的                                   | 例                                          |
|--------------------|----------------------------------------|--------------------------------------------|
| <b>MaterialApp</b> | ナビゲーション, テーマ, ローカライゼーションを提供するルートウィジェット | <code>return MaterialApp(...)</code>       |
| <b>Scaffold</b>    | AppBar や body などを含むメイン画面の構造を定義する       | <code>return Scaffold(...)</code>          |
| <b>AppBar</b>      | アプリのタイトルや操作を表示する上部ナビゲーションバー            | <code>AppBar(title: Text('My App'))</code> |
| <b>Text</b>        | 画面にシンプルなテキストを表示する                      | <code>Text('Hello, world!')</code>         |

- ポイント:
  - ◇ Flutter では,build() 内で return を使ってウィジェットを返さないと画面に表示されない.
  - ◇ return がないと,Flutter は何を表示すべきか判断できない.

c. 比較: Java における return の使われ方

- Java も Flutter (Dart) も,return を使って作成したオブジェクトを返す.
- Java では return は View や UI コンポーネントを返すことが多い.
- Flutter では,return はよりシンプルでウィジェット中心の設計になっている.
- 主な違い:

| 機能     | Java (Android)                | Flutter (Dart)                |
|--------|-------------------------------|-------------------------------|
| 戻り値の型  | View (TextView,Button など) を返す | Widget (Text,Scaffold など) を返す |
| UI の構造 | UI は通常 XML で管理                | Dart 内でウィジェットを使って UI を構築      |
| 柔軟性    | View の階層を変更する必要がある            | build() メソッド内で簡単に管理可能         |

ステップ 8 : Flutter の MaterialApp とは ?

a. MaterialApp とは ?

- MaterialApp は,Google の Material Design に準拠した Flutter アプリの基盤となる.
- アプリの基本構造とビジュアルスタイルを提供する.
- テーマ,ナビゲーション,デフォルトの動作を設定し,スムーズなユーザー体験を実現する.